

Programmation Linéaire

TP n°2 : Cplex Callable Library

Utilisation de la bibliothèque d'optimisation Cplex

Cplex inclut une bibliothèque, développée en plusieurs langages (C : `libcplex.a`, C++ : `libilocplex.a`, Java : `cplex.jar`, .Net, ...), et qui contient un certain nombre de fonctions qu'il est possible d'appeler dans un programme (C, C++, Java, ...). Dans le cadre de ce TP, nous travaillerons exclusivement en C.

1. Utilisation de la bibliothèque Ilog Cplex

1.1. Description

La bibliothèque Ilog Cplex contient des routines organisées en plusieurs catégories :

1. les routines d'*optimisation* qui permettent de résoudre un problème ;
2. les routines de *modélisation* qui permettent de (re)définir un problème ;
3. les routines de *paramétrisation* qui servent à connaître ou à modifier les valeurs des paramètres des fonctions de Cplex ;
4. les routines d'*interrogation* qui permettent d'accéder aux informations inhérentes à un problème et à sa solution.

1.2. Documentation technique

1.2.1 Organisation et conventions de nommage

Les structures et fonctions Cplex ont pour préfixe "CPX". Les fichiers d'entête (essentiellement, **cplex.h** pour la bibliothèque C) contiennent les constantes (ex. : `CPX_INFBOUND` pour désigner l'infini) et les prototypes de toutes les routines et paramètres de Cplex. Ces fichiers sont situés dans le répertoire : `/LOCAL/CPLEX_Studio129/cplex/include/ilcplex/`

1.2.2 Compilation

Pour faire appel aux objets Cplex, il faut bien évidemment :

1. dans vos sources, penser à inclure **cplex.h** : `#include<cplex.h>`
2. dans la commande de compilation, indiquer les chemins (fichier d'entêtes et bibliothèque) et le lien à la bibliothèque (NB : il faut aussi utiliser les librairies standard **m**, **pthread** et **dl**) :

```
-I/LOCAL/CPLEX_Studio129/cplex/include/ilcplex  
-L/LOCAL/CPLEX_Studio129/cplex/lib/x86-64_linux/static_pic  
-lcplex -lm -lpthread -ldl
```

1.2.3 Initialisation de l'environnement Cplex

Pour invoquer des objets ou fonctions Cplex, il faut en premier lieu initialiser l'environnement, Cplex ayant besoin de quelques structures de données internes pour opérer. La routine **CPXopenCplex()** permet cette initialisation et retourne un pointeur vers l'environnement créé (dont le type est **CPXENVptr**). Toutes les structures et fonctions Cplex nécessitent la référence à un environnement. Notons que plusieurs environnements (autant que de licences disponibles) peuvent être ouverts simultanément. Le mode d'emploi de **CPXopenCplex()** est le suivant :

```
CPXENVptr CPXPUBLIC CPXopenCplex (int *statut);  
CPXENVptr : type Cplex qui désigne un pointeur vers l'environnement  
statut : contient le code de l'erreur éventuelle (0 en cas de succès).
```

Ainsi, pour ouvrir l'environnement Cplex, il faut déclarer une variable de type **CPXENVptr**, déclarer une variable de type **int**, puis affecter l'environnement par appel à **CPXopenCplex()** :

```
CPXENVptr env = NULL; int statut = 0;  
env = CPXopenCplex (&statut);
```

1.2.4 Gestion de l'erreur

La plupart des fonctions retournent un entier : 0 en cas de succès, une certaine constante symbolique en cas d'erreur. Pour retrouver la description associée à un code numérique donné, il faut déclarer une chaîne de caractères de taille suffisante, puis faire appel à la fonction **CPXgeterrorstring** ; le message correspondant à l'entier **statut** est écrit dans une chaîne (**str_err_message** dans l'exemple ci-suit) :

```
str_err_message[1024];  
CPXgeterrorstring (env, statut, str_err_message);
```

1.2.5 Libération de l'espace mémoire

Il faut libérer l'espace réservé par les structures Cplex. Malheureusement, il n'existe pas de fonction permettant de libérer toute réservation mémoire faite pour des structures liées à un certain environnement ; il faut donc *d'abord* libérer les différentes structures (notamment, les programmes linéaires par la fonction **CPXfreeprob()**), puis l'environnement Cplex par la fonction **CPXcloseCPLEX()** :

```
// libérer l'espace occupé par un problème  
int CPXPUBLIC CPXfreeprob (CPXCENVptr env, CPXLPptr *lp_p);  
  
// libérer l'espace occupé par un environnement  
int CPXPUBLIC CPXcloseCPLEX (CPXENVptr *env_p);
```

2. Manipulation d'un programme linéaire

2.1. Définition

Une fois un environnement ouvert, il faut initialiser une structure de donnée dans laquelle un problème à résoudre sera stocké. Cette initialisation se fait par appel à la routine **CPXcreateprob()**, qui retourne un pointeur vers un PL :

<pre>CPXLPptr CPXPUBLIC CPXcreateprob(CPXCENVptr env, int* status_p, const char*& probname_str);</pre> CPXLPptr : type Cplex qui désigne un pointeur vers une structure PL ; probname_str : désigne le nom donné au problème.

Plusieurs problèmes peuvent être ouverts simultanément dans un même environnement. Après son initialisation, il est possible de définir un problème de plusieurs manières : par structures informatiques, par appels successifs à des fonctions de modification de PL, ou encore, par lecture de fichier.

2.2. Initialisation

2.2.1 Par structures

On peut rassembler toutes les données dans différents tableaux et appeler la routine **CPXcopylp()** qui va copier les données dans la structure de donnée PL :

<pre>int CPXPUBLIC CPXcopylp (CPXCENVptr env, CPXLPptr lp, int numcols, int numrows, int objsen, const double *obj, const double *rhs, const char *sense, const int *matbeg, const int *matent, const int *matind, const double *matval, const double *lb, const double *ub, const double *rngval);</pre> numcols, numrows, objsen : nombre de colonnes, nombre de lignes, sens de l'objectif ($\in \{\text{CPX_MIN}, \text{CPX_MAX}\}$). obj, rhs, sense : vecteur objectif, vecteur membre droit, vecteur de sens des contraintes ($\in \{\text{'L'}, \text{'E'}, \text{'G'}\}$). matbeg, matent, matind, matval : vecteurs représentant la matrice des contraintes. lb, ub : vecteurs des bornes inférieures et supérieures pour les variables. rngval : pas dit dans ce TP.

Seuls les coefficients *non nuls* de la matrice des contraintes sont représentés, dans le tableau **matval** ; **matbeg** associe à chaque colonne de la matrice des contraintes l'indice du premier élément à lire dans **matval** ; **matind** associe à chaque coefficient de **matval** son indice (i.e., sa ligne) dans la colonne à laquelle il appartient ; enfin, **matent** associe à chaque colonne le nombre de coefficients non nuls qu'elle contient. Ainsi, les coefficients non nuls de la colonne j sont :

matval[matbeg[j]], matval[matbeg[j]+1], ..., matval[matbeg[j]+matcnt[j]-1]

et se situent respectivement dans la ligne :

matind[matbeg[j]], matind[matbeg[j] + 1], ..., matind[matbeg[j] + matcnt[j]-1]

2.2.2 Par fonctions

On peut appeler successivement les routines **CPXnewcols()**, **CPXnewrows()**, **CPXaddcols()**, **CPXaddrows()** : soit l'on définit le problème par ses variables (**CPXnewcols()**) puis on remplit la matrice en décrivant totalement les contraintes (**CPXaddrows()**), soit l'on définit le problème par ses contraintes (**CPXnewrows()**) puis on remplit la matrice en décrivant totalement les variables (**CPXaddcols()**).

2.2.3 Par lecture de fichier

Si les données sont déjà rangées dans un fichier aux formats **lp** ou **mps**, la routine **CPXreadcopyprob()** permet de lire le fichier et d'en copier les données dans la structure PL :

```
int CPXreadcopyprob(CPXENVptr env, CPXLPptr lp, char *nom_fichier, char *type_fichier);
```

2.3. Interrogation

Plusieurs accesseurs sont naturellement définis qui permettent de récupérer les données d'un problème, les plus importants étant sans doute ceux permettant d'en connaître la taille :

```
// pour récupérer le nombre de variables de lp
int CPXPUBLIC CPXgetnumcols (CPXCENVptr env, CPXCLPptr lp);
// pour récupérer le nombre de contraintes de lp
int CPXPUBLIC CPXgetnumrows (CPXCENVptr env, CPXCLPptr lp);
```

Pour les autres accesseurs (objectif, membre droit, lignes, colonnes, bornes, ...), se reporter à l'aide Cplex (ne serait-ce, que **cplex.h**).

2.4. Modification d'un problème

2.4.1 Ajout de ligne (contrainte), de colonne (variable)

Création de lignes et de colonnes incomplètes

Par incomplétude, on entend absence des coefficients correspondants dans la matrice des contraintes. Une contrainte est alors caractérisée par le membre droit et le sens, tandis qu'une variable est caractérisée par son coefficient dans la fonction objectif, son type (continu, binaire, ...) et ses bornes (inférieure et supérieure). Les nouvelles lignes ou colonnes créées sont ajoutées à la suite (du point de vue de leur indice) des lignes et colonnes déjà existantes.

```
int CPXPUBLIC CPXnewrows      // ajout de contraintes vides à un problème
(
    CPXCENVptr env, CPXLPptr lp,
    int nb, const double *rhs, const char *sense, const double *rngval, char **rowname
);
nb :                nombre de contraintes
rhs :               vecteur (de taille nb) des membres droits
const char *sense : vecteur (de taille nb) des sens des contraintes
```

```

int CPXPUBLIC CPXnewcols      // ajout de variables à un problème
(
    CPXCENVptr env, CPXLPptr lp,
    int nb, const double *obj, const double *lb, const double *ub, const char *xctype, char **colname
);
nb :          nombre de variables
obj :         vecteur (de taille nb) des coefficients objectif
lb, ub :      vecteurs (de taille nb) des bornes inf. et sup.
const char *xctype vecteur des types des variables (réel == 'C', binaire == 'B', entier == 'I', ...)

```

Une ligne (*resp.*, une colonne) ainsi créée peut ensuite être complétée par appel à la fonction **CPXaddcols()** (*resp.*, **CPXaddrows()**), ou encore, aux fonctions de modification des coefficients, telles que **CPXchgcoef()** ou **CPXchgcoeflist()**.

Création de lignes et de colonnes complètes

Il faut donner les mêmes informations que précédemment, avec en plus la description des coefficients de la matrice. Les fonctions sont **CPXaddrows()** et **CPXaddcols()**. De nouveau, les lignes ou colonnes insérées sont ajoutées à la suite (du point de vue de leur indice) des lignes et colonnes déjà existantes. **Attention** : autant l'on peut ajouter implicitement une variable (donc, une colonne) en ajoutant une ligne (cas d'une contrainte faisant intervenir une variable qui n'était jusque lors impliquée dans aucune contrainte existante), autant l'on ne peut ajouter de contrainte (considérer un indice de ligne supplémentaire) au moment de l'ajout d'une variable.

```

int CPXaddrows      // ajout de lignes complètes à un problème
(
    CPXCENVptr env, CPXLPptr lp,
    int cnt, int rcnt, int nzcnt, const double *rhs, const char *sense,
    const int *rmatbeg, const int *rmatind, const double *rmatval,
    char **colname, char **rowname
);
ccnt :    nombre de contraintes ajoutées
rcnt :    nombre de variables (éventuellement) ajoutées
nzcnt :   nombre de coefficients non nuls à insérer dans la matrice

```

```

int CPXaddcols      // ajout de colonnes complètes à un problème
(
    CPXCENVptr env, CPXLPptr lp,
    int cnt, int nzcnt, const double *obj
    const int *cmatbeg, const int *cmatind, const double *cmatval,
    const double *lb, const double *ub, char **colname
);

```

2.4.2 Suppression de ligne (contrainte), de colonne (variable)

Les fonctions **CPXdclrows** et **CPXdclsetrows** (*resp.*, **CPXdclcols** et **CPXdclsetcols**) permettent de supprimer un ensemble de lignes (*resp.*, de colonnes) :

int CPXdelrows(CPXCENVptr env, CPXLPptr lp, int begin, int end) int CPXdelcols(CPXCENVptr env, CPXLPptr lp, int begin, int end) begin, end : indice des première et dernière lignes/colonnes à supprimer
int CPXdelsetrows(CPXCENVptr env, CPXLPptr lp, int *delstat) int CPXdelsetcols(CPXCENVptr env, CPXLPptr lp, int *delstat) delstat : vecteur de taille CPXgetnumrows(env,lp)/CPXgetnumcols(env,lp) ; les lignes / colonnes d'indice ind tel que delstat[ind] == 1 sont supprimées

2.4.3 Modification de données du problème

Un ensemble de fonctions préfixées par **CPXchg** (principalement, **CPXchgcoef**, **CPXchgcoefficient**, **CPXchgbd**, **CPXchgobj**, **CPXchgrrhs**, **CPXchgrrhsense**, **CPXchgrrhsobj**, **CPXchgrrhsobjtype**) permettent de modifier :

1. les coefficients de la matrice, incluant ceux de la fonction objectif et du membre droit ;
2. les bornes sur les variables ;
3. le sens des contraintes ;
4. les propriétés du problème.

Fonctions **CPXchgcoef**, **CPXchgcoefficient** :

int CPXPUBLIC CPXchgcoef	// (re)définition d'un coefficient de la matrice
(	
CPXCENVptr env, CPXLPptr lp, int i, int j, double newvalue	
);	
i :	indice de ligne (-1 pour l'objectif)
j :	indice de colonne (-1 pour le membre droit)
newvalue :	la valeur du coefficient

int CPXPUBLIC CPXchgcoefficient	// (re)définition d'un ensemble de coefficients de la matrice
(	
CPXCENVptr env, CPXLPptr lp,	
int numcoefs, const int *rowlist, const int *collist, const double *vallist	
);	
numcoefs :	nombre de coefficients à modifier
rowlist :	vecteur des numcoefs indices de ligne des coefficients à modifier
collist :	vecteur des numcoefs indices de colonne des coefficients à modifier
vallist :	vecteur des numcoefs nouveaux coefficients à affecter

Pour les autres fonctions, se reporter à la documentation Cplex.

3. Résolution d'un problème

La résolution du problème se fait à l'aide d'une des routines d'optimisation suivantes (après que le problème ait été défini et initialisé !) :

fonction	équivalent	Synopsis
CPXlpopt()	optimize	int CPXPUBLIC CPXlpopt (CPXCENVptr env, CPXLPptr lp);
CPXprimopt()	primopt	int CPXPUBLIC CPXprimopt (CPXCENVptr env, CPXLPptr lp);
CPXdualopt()	tranopt	int CPXPUBLIC CPXdualopt (CPXCENVptr env, CPXLPptr lp);

3.1. Récupération de la solution

Après appel à une fonction d'optimisation, les informations relatives à la résolution d'un problème peuvent être récupérées par la routine **CPXsolution()** :

```
int CPXPUBLIC CPXsolution
(
    CPXCENVptr env, CPXCLPptr lp, int *lpstat_p,
    double *objval_p, double *x, double *pi, double *slack, double *dj
);
x, pi :      variables primales, variables duales
dj, slack :  coûts réduits des variables primales, variables d'écart
```

4. Sauvegarde des informations

Les fonctions **CPXwriteprob** et **CPXsolwrite** permettent respectivement d'écrire un problème et la solution d'un problème dans un fichier :

```
int CPXPUBLIC CPXwriteprob      /      int CPXPUBLIC CPXsolwrite
    (CPXCENVptr env, CPXCLPptr lp, const char *filename_str, const char *filetype_str)
filetype_str :      type du fichier (nulle si l'extension est donnée dans le nom du fichier)
```

Annexes

A.1. Compilation

1. $\text{exo.c} \rightarrow \text{exo.o}$
`gcc -c exo.c -o exo.o -I/LOCAL/CPLEX_Studio129/cplex/include/ilcplex`
2. $\text{exo.o} \rightarrow \text{exo}$
`gcc exo.o -o exo -L/LOCAL/CPLEX_Studio129/cplex/lib/x86-64_linux/static_pic
-lcplex -lm -lpthread -ldl`

A.2. Programme Cplex minimal

```

#include<cplex.h>

int main()
{
    // declarations
    CPXENVptr env = NULL; // environnement Cplex
    CPXLPptr lp = NULL; // PL Cplex
    int statut = 0; // statut technique des fonctions

    // ouverture de Cplex
    env = CPXopenCPLEX (&statut);

    // creation d'un PL
    lp = CPXcreateprob(env, &statut, "gros_probleme");

    // instanciation d'un PL
    statut = CPXreadcopyprob(env, lp, "mon_pb.lp", NULL);

    // resolution d'un PL
    statut = CPXprimopt(env, lp);

    // sauvegarde de la solution dans un fichier
    statut = CPXsolwrite(env, lp, "resolution.txt");

    // liberation de l'espace memoire
    statut = CPXfreeprob (env, &lp);
    statut = CPXcloseCPLEX (&env);

    // on s'en va !
    return 0;
}

```

Sujet :

On considère les problèmes suivants :

$$(P_1) \left\{ \begin{array}{lcl} \max & 5x_1 & - 3x_2 \\ \text{s.c.} & x_1 & - x_2 \geq 2 \\ & 2x_1 & + 3x_2 \leq 6 \\ & -x_1 & + 5x_2 \leq 10 \\ & x_1 & , \quad x_2 \geq 0 \end{array} \right. \quad (P_3) \left\{ \begin{array}{lcl} \max & x_1 & + 2x_2 + 3x_3 \\ \text{s.c.} & -x_1 & + x_2 + x_3 \leq 20 \\ & x_1 & - 3x_2 + x_3 \leq 30 \\ & x_1 & \leq 40 \\ & x_1 & , \quad x_2 \geq 0 \end{array} \right.$$
$$(P_2) \left\{ \begin{array}{lcl} \min & x & \\ \text{s.c.} & x & \leq 200 \end{array} \right.$$

Exercice 1

- Écrire un programme C qui, étant donné un fichier au format **lp** passé en paramètre :
 - résout le programme linéaire décrit dans le fichier ;
 - affiche les informations correspondantes (statut, solutions optimales primale et duale, variables d'écart, coûts réduits).
- Tester le programme sur (P_1) , (P_2) et (P_3) .
- Améliorer le programme en définissant une structure **resolution** et son interface : création, destruction, affichage.
- Penser à gérer l'erreur !

Exercice 2

- Ajouter comme paramètre du programme le choix d'une méthode d'optimisation (algorithme primal ou dual du simplexe).
- Enrichir le programme des fonctionnalités suivantes, à la demande de l'utilisateur (cela induit l'introduction d'un menu) :
 - ajout de contrainte(s) à un problème chargé ;
 - ajout de variable(s) à un problème chargé ;
 - modification de coefficients de la fonction objectif, du membre droit, de la matrice d'un problème chargé ;
 - résolution et affichage de la solution d'un problème chargé ;
 - sauvegarde d'un problème chargé ;
 - sauvegarde de la solution d'un problème chargé et résolu.

Exercice 3

Écrire un programme C permettant de résoudre le programme linéaire suivant :

$$\left\{ \begin{array}{ll} \text{Max} & -5x_1 - 7x_2 + 8x_3 \\ \text{s.c.} & 3x_1 + 2x_2 \leq 9 \\ & 10x_1 + 2x_3 \geq -11 \\ & x_1 \leq 0 \\ & 1 \leq x_2 \leq 4 \\ & 0 \leq x_3 \leq 3 \end{array} \right.$$

Vous utiliserez pour cela la Cplex Callable Library et en particulier la fonction `CPXcopylp`.

Exercice 4 : Compagnie aérienne

Une compagnie aérienne veut ajouter des vols au départ et à destination de l'aéroport qui lui sert de plateforme (un hub). Ainsi, elle doit engager du personnel mais elle s'interroge sur le nombre de recrutements nécessaires sachant qu'un effectif fourni permet d'accroître le niveau de satisfaction des clients.

Pour un niveau de satisfaction donné, on connaît le nombre minimal d'employés nécessaires pour chaque tranche horaire de la journée. Il existe 5 types de services quotidiens possibles pour un employé. Pour chacun de ces services de 8 heures, l'entreprise a déjà fixé les horaires de travail et la rémunération correspondante pour une journée de travail d'un employé.

Période	Services					Nb min d'emp.
	1	2	3	4	5	
06h-08h	x					48
08h-10h	x	x				79
10h-12h	x	x				65
12h-14h	x	x	x			87
14h-16h		x	x			64
16h-18h			x	x		73
18h-20h			x	x		82
20h-22h				x		43
22h-00h				x	x	52
00h-06h					x	15
Coût par emp. par j.	170€	160€	175€	180€	195€	

L'entreprise veut savoir comment gérer son personnel de manière à satisfaire ses besoins avec un coût minimal et par conséquent connaître le nombre total d'employés et le coût optimal correspondant.

Exercice 5

Le problème concerne un restaurant ouvert sept jours sur sept. L'expérience antérieure démontre que le nombre d'employés nécessaires pour chaque jour de la semaine est :

Jour	Lundi	Mardi	Mercredi	Jeudi	Vendredi	Samedi	Dimanche
Effectif	14	13	15	16	19	18	11

Le cycle de travail de chaque employé consiste en 5 jours consécutifs de travail et 2 de repos.

1. Formuler le problème qui consiste à minimiser le nombre total de travailleurs. Quel est cet effectif ?

2. Quel effet ont chacune des modifications suivantes sur le nombre total d'employés ?
 - (a) Le nombre d'employés passe de 16 à 18 le jeudi.
 - (b) Le nombre d'employés passe de 14 à 11 le lundi.
3. Chaque travailleur est payé 1000 Euros par mois. La fonction objectif du problème formulé au 1. peut être interprétée comme une dépense globale minimum.
De combien peut varier le nombre de travailleurs du mercredi sans perturber cette dépense ?
4. La tâche de l'équipe commençant le jeudi étant plus lourde, le gestionnaire du restaurant considère que ses employés doivent être rémunérés 1100 Euros par mois. Cela a-t-il un effet sur la solution optimale ? Sur la dépense globale ?
5. Le gestionnaire se demande également s'il ne faut pas diminuer à 900 Euros par mois le salaire de l'équipe qui prend son service le lundi. Cela a-t-il un effet sur la solution optimale ? Sur la dépense globale ?
6. Le gestionnaire propose l'introduction d'une équipe supplémentaire dont les jours de repos seraient le mardi et le dimanche. Ces derniers n'étant pas consécutifs, il décide de fixer le salaire des employés de cette nouvelle équipe à 1200 Euros. Cette proposition a-t-elle un effet sur la dépense globale ?